

1.Create a class BankAccount in Python with private attributes
__accountno,__name, __balance.

Add

parameterized constructor

methods:

deposit(amount)

withdraw(amount)

set_accountno

get_accountno

set_name

get_name

get_balance()

set_balance()

Code:

```
class BankAccount:
    def __init__(self, accountno, name, balance):
        self.__accountno = accountno
        self.__name = name
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited: ₹{amount}")
        else:
            print("Invalid deposit amount")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrew: ₹{amount}")
        else:
            print("Insufficient balance or invalid amount")

    def set_accountno(self, accountno):
        self.__accountno = accountno

    def get_accountno(self):
        return self.__accountno

    def set_name(self, name):
        self.__name = name

    def get_name(self):
        return self.__name
```

```

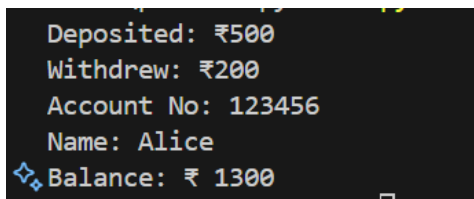
    def set_balance(self, balance):
        if balance >= 0:
            self.__balance = balance

    def get_balance(self):
        return self.__balance

if __name__ == "__main__":
    acc = BankAccount("123456", "Alice", 1000)
    acc.deposit(500)
    acc.withdraw(200)
    print("Account No:", acc.get_accountno())
    print("Name:", acc.get_name())
    print("Balance: ₹", acc.get_balance())

```

Output:



```

Deposited: ₹500
Withdrew: ₹200
Account No: 123456
Name: Alice
❖ Balance: ₹ 1300

```

2.How will you define a static method in Python?Explore and give an example.

Code:

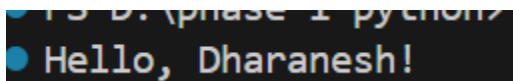
```

class Utility:
    @staticmethod
    def greet(name):
        return f"Hello, {name}!"

print(Utility.greet("Dharanesh"))

```

Output:



```

● 13 D. (phase 1 python)
● Hello, Dharanesh!

```

3.Give examples for dunder methods in Python other than `__str__` and `__init__` .

Code:

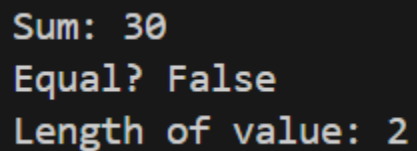
```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __eq__(self, other):
        return self.value == other.value

    def __len__(self):
        return len(str(self.value))

if __name__ == "__main__":
    a = Number(10)
    b = Number(20)
    c = a + b
    print("Sum:", c.value)
    print("Equal?", a == b)
    print("Length of value:", len(c))
```

Output:

```
Sum: 30
Equal? False
Length of value: 2
```

4. Explore some supervised and unsupervised models in ML.**What is Supervised Learning?**

Supervised learning is a type of machine learning where the model is trained using **labeled data**. That means the input data comes with the correct output (target/label), and the model learns to map inputs to outputs.

Key Points:

- Uses labeled data.
- The goal is to **predict** or **classify** future data.
- Common tasks: **Classification** and **Regression**.

Examples:

- Linear Regression - Predicting house prices.
- Logistic Regression - Predicting whether a transaction is fraudulent.
- Decision Trees
- Support Vector Machines (SVM)
- K-Nearest Neighbors (KNN)

What is Unsupervised Learning?

Unsupervised learning is a type of machine learning where the model is trained using **unlabeled data**. The system tries to learn the patterns and structure in the data without predefined labels.

Key Points:

- No labeled output is provided.
- The goal is to **discover patterns, clusters, or relationships** in the data.
- Common tasks: **Clustering** and **Dimensionality Reduction**.

Examples:

- K-Means Clustering - Grouping customers based on purchasing behavior.
- Hierarchical Clustering
- Principal Component Analysis (PCA) - Reducing feature dimensions.
- DBSCAN (Density-Based Spatial Clustering)

5.Implement Stack with class in Python.

Code:

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)
        print(f"Pushed: {item}")

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return "Stack is empty"
```

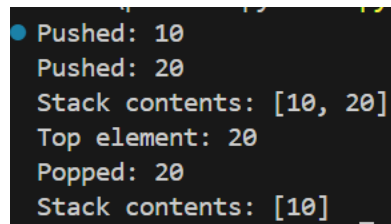
```
def peek(self):
    if not self.is_empty():
        return self.stack[-1]
    return "Stack is empty"

def is_empty(self):
    return len(self.stack) == 0

def display(self):
    print("Stack contents:", self.stack)

if __name__ == "__main__":
    s = Stack()
    s.push(10)
    s.push(20)
    s.display()
    print("Top element:", s.peak())
    print("Popped:", s.pop())
    s.display()
```

Output:



```
Pushed: 10
Pushed: 20
Stack contents: [10, 20]
Top element: 20
Popped: 20
Stack contents: [10]
```